

pythonTM

pandas

Why Pandas?

- It is a package useful for data analysis and manipulation.
- Pandas provide an easy way to create, manipulate and wrangle the data.
- Pandas provide powerful and easy-to-use data structures, as well as the means to quickly perform operations on these structures.

Pandas deals with 3 data structures -

- 1.Series
- 2.Data Frame
- 3.Panel

Series

Series is a one-dimensional array like structure with homogeneous data, which can be used to handle and manipulate data. What makes it special is its index attribute, which has incredible functionality and is heavily mutable.

Index	Data
0	10
1	15
2	18
3	22

Syntax to create a Series:

<Series Object>=pandas.Series (data, index=idx (*optional*))

```
import pandas as pd
import numpy as np
arr=np.array([10,15,18,22])
s = pd.Series(arr)
print(s)
```

Output-

0 10

1 15

2 18

3 22

Creating a series from Scalar Values

```
1 import pandas as pd
2 s = pd.Series(50, index=[0, 1, 2, 3, 4])
3 print(s)
4
```

```
0    50
1    50
2    50
3    50
4    50
dtype: int64
```

Creating a series from a Dictionary

```
1  # import the pandas lib as pd
2  import pandas as pd
3
4  # create a dictionary
5  d = {'Name' : 'Hardik', 'Iplteam' : 'MI', 'Runs' : 1500}
6
7  # create a series
8  s = pd.Series(d)
9
10 print(s)
11
```

```
Name      Hardik
Iplteam    MI
Runs      1500
dtype: object
```

Mathematical Operations in Series

```
import pandas as pd
s1=pd.Series([1,2,3,4,5],index=['a','b','c','d','e'])
s2=pd.Series([10,20,30,40,50],index=['a','b','c','d','e'])
s3=pd.Series([5,14,23,32],index=['a','b','c','d'])
print('To Add Series1 & series2')
print('-----')
print(s1+s2)
print('To Add Series2 & Series3')
print('-----')
print(s2+s3)
print('To Add Series2 & series3 and Filled Non Matching Index with 0')
print('-----')
print(s2.add(s3,fill_value=0))
```

OUTPUT :

```
To Add Series1 & series2
```

```
-----  
a    11  
b    22  
c    33  
d    44  
e    55
```

```
dtype: int64
```

```
To Add Series2 & Series3
```

```
-----  
a    15.0  
b    34.0  
c    53.0  
d    72.0  
e     NaN
```

```
dtype: float64
```

```
To Add Series2 & series3 and Filled Non Matching Index with 0
```

```
-----  
a    15.0  
b    34.0  
c    53.0  
d    72.0  
e    50.0
```

```
dtype: float64
```


head ()

It is used to access the **first 5 rows of a series**.

Note : To access first 3 rows we can call **series_name.head(3)**

```
: 1 import pandas as pd
   2 import numpy as np
   3 arr=np.array([10,15,18,22,55,77,42,48,97])
   4 # create a series from array
   5 s = pd.Series(arr)
   6 # to print first 5 rows
   7 print (s.head())
   8 # To print first 3 rows
   9 print(s.head(3))
```

0 10

1 15

2 18

3 22

4 55

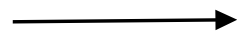
dtype: int32

0 10

1 15

2 18

dtype: int32



Result of **s.head()**



Result of **s.head(3)**

tail ()

It is used to access the **last 5 rows of a series**.

Note : To access last 4 rows we can call **series_name.tail (4)**

```
1 import pandas as pd
2 import numpy as np
3 arr=np.array([10,15,18,22,55,77,42,48,97])
4 # create a series from array
5 s = pd.Series(arr)
6 # to print last 5 rows
7 print (s.tail())
8 # To print last 4 rows
9 print(s.tail(4))
```

```
4    55
5    77
6    42
7    48
8    97
dtype: int32
5    77
6    42
7    48
8    97
dtype: int32
```

Selection in Series

1. **loc** index label :

```
1 import pandas as pd
2 import numpy as np
3 arr=np.array([10,15,18,22,55,77])
4 s = pd.Series(arr)
5 print(s)
6 print(s.loc[:2])
7 print(s.loc[3:4])
8 s.loc[2:3]
```



```
0    10
1    15
2    18
3    22
4    55
5    77
```

dtype: int32

```
0    10
1    15
2    18
```

dtype: int32

```
3    22
4    55
```

dtype: int32

```
2    18
3    22
```

dtype: int32

2. Selection Using **iloc** index label :

```
1 import pandas as pd
2 import numpy as np
3 arr=np.array([10,15,18,22,55,77])
4 s = pd.Series(arr)
5 print(s)
6 print(s.iloc[:2]) → 
7 print(s.iloc[3:4])
8 s.iloc[2:3]
```

```
0    10
1    15
2    18
3    22
4    55
5    77
```

dtype: int32

```
0    10
1    15
```

dtype: int32

```
3    22
```

dtype: int32

```
2    18
```

dtype: int32

3. Selection Using `[ ]` :

```
1 import pandas as pd
2 import numpy as np
3 arr=np.array([10,15,18,22,55,77])
4 s = pd.Series(arr)
5 print(s)
6 print(s[1])
7 print('\n')
```

```
0    10
1    15
2    18
3    22
4    55
5    77
dtype: int32
15
```

```
3    22
dtype: int32
```

```
0    10
1    15
2    18
dtype: int32
```

Slicing in Series

```
1 import pandas as pd
2 import numpy as np
3 arr=np.array([10,15,18,22,55,77])
4 s = pd.Series(arr,index=['A','B','C','D','E','F'])
5 print(s)
6 print(s[1:5:2])
7 print(s[0:6:2])
8
```

```
A    10
B    15
C    18
D    22
E    55
F    77
dtype: int32
B    15
D    22
dtype: int32
A    10
C    18
E    55
dtype: int32
```

DATAFRAME

It is a two-dimensional object that is useful in representing data in the form of rows and columns.

COLUMNS			
PLAYERNAME			
IPLTEAM			
BASEPRICEINCR			
0	ROHIT	MI	13
1	VIRAT	RCB	17
2	HARDIK	MI	14

INDEX

A data frame can be created using any of the following-

- 1. Series**
- 2. Lists**
- 3. Dictionary**
- 4. A numpy 2D array**

How to create Dataframe From Series

Program-

```
import pandas as pd  
s = pd.Series(['a','b','c','d'])  
df=pd.DataFrame(s)  
print(df)
```

OUTPUT



0	
0	a
1	b
2	c
3	d

Default Column Name As 0

DataFrame from Dictionary of Series (with default Index)

```
import pandas as pd
name=pd.Series(['Hardik','Virat'])
team=pd.Series(['MI','RCB'])
dic={'Name':name,'Team':team}
df=pd.DataFrame(dic)
print(df)
```

	Name	Team
0	Hardik	MI
1	Virat	RCB

DataFrame from List of Dictionaries

```
1 import pandas as pd
2 l = [{'Name': 'Sachin', 'SirName': 'Bhardwaj'},
3      {'Name': 'Vinod', 'SirName': 'Verma'},
4      {'Name': 'Rajesh', 'SirName': 'Mishra'}]
5 df1=pd.DataFrame(l)
6 print(df1)
```

	Name	SirName
0	Sachin	Bhardwaj
1	Vinod	Verma
2	Rajesh	Mishra

SELECTING SPECIFIC COLUMN

```
import pandas as pd

data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'],
        'Age':[27, 24, 22, 32],
        'Address':['Delhi', 'Kanpur', 'Allahabad', 'Kannauj'],
        'Qualification':['Msc', 'MA', 'MCA', 'Phd']}

df = pd.DataFrame(data)

print(df[['Name', 'Qualification']])
```

Dataframe with a List Of Dictionaries, Row Indices, and Column Indices.

```
import pandas as pd
data = [{'a': 1, 'b': 2}, {'a': 5, 'b': 10, 'c': 20}]

# With two column indices, values same as dictionary keys
df1 = pd.DataFrame(data, index=['first', 'second'], columns=['a','b'])

# With two column indices with one index with other name
df2 = pd.DataFrame(data, index=['first','second'], columns=['a','b1'])
print(df1)
print(df2)
```

OUTPUT

		a	b
first	1	2	
second	5	10	

		a	b1
first	1	NaN	
second	5	NaN	

Accessing Row Labels of DataFrame

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Steve', 'Lia', 'Vin', 'Katie'],
    'Age': [32, 28, 45, 38],
    'Gender': ['Male', 'Female', 'Male', 'Female'],
    'Rating': [3.45, 4.6, 3.9, 2.78]},
    index=['r1', 'r2', 'r3', 'r4'])

result = df.index
print('Output Accessed Row Labels:', result)
```

OUTPUT

```
Output Accessed Row Labels:
Index(['r1', 'r2', 'r3', 'r4'], dtype='object')
```

Accessing Column Labels of DataFrame

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Steve', 'Lia', 'Vin', 'Katie'],
    'Age': [32, 28, 45, 38],
    'Gender': ['Male', 'Female', 'Male', 'Female'],
    'Rating': [3.45, 4.6, 3.9, 2.78]},
    index=['r1', 'r2', 'r3', 'r4'])

result = df.columns
print('Output Accessed column Labels:', result)
```

OUTPUT

```
Output Accessed column Labels:
Index(['Name', 'Age', 'Gender', 'Rating'], dtype='object')
```

Slicing a DataFrame Object

1) SLICING A DATAFRAME BY POSITION

Syntax :

```
DataFrame.iloc[row_start:row_end, column_start:column_end]
```

```
df = pd.DataFrame([[ 'a', 'b'], [ 'c', 'd'], [ 'e', 'f'], [ 'g', 'h']],  
columns=[ 'col1', 'col2'])  
print(df)  
result = df.iloc[1:3, :]  
print(result)
```

OUTPUT

Input DataFrame:

	col1	col2
0	a	b
1	c	d
2	e	f
3	g	h

Output:

	col1	col2
1	c	d
2	e	f

2) SLICING DATAFRAME BY LABEL USING

Syntax :

```
DataFrame.loc[rowlabel_start:rowlabel_end, columnlabel_start:columnlabel_end]
```

```
df = pd.DataFrame([[ 'a', 'b'], [ 'c', 'd'], [ 'e', 'f'], [ 'g', 'h']],  
                  columns=[ 'col1', 'col2'],  
                  index=[ 'r1', 'r2', 'r3', 'r4'])  
print(df)  
result = df.loc['r1': 'r3', 'col1']  
print(result)
```

OUTPUT

Original DataFrame:

	col1	col2
r1	a	b
r2	c	d
r3	e	f
r4	g	h

Output:

```
r1    a  
r2    c  
r3    e  
Name: col1,  
dtype: object
```


Renaming a column or row label

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
print(df)

# Rename column 'A' to 'AAA'
df = df.rename(columns={'A': 'AAA'})

print("Modified DataFrame:")
print(df)
```

OUTPUT

Original DataFrame:

	A	B
0	1	4
1	2	5
2	3	6

Modified DataFrame:

	AAA	B
0	1	4
1	2	5
2	3	6

Adding a New Column

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})  
print("Original DataFrame\n")  
  
df['C'] = [7, 8, 9]  
print("DataFrame after adding new column 'C':")  
print(df)
```

OUTPUT

Original DataFrame

	A	B
0	1	4
1	2	5
2	3	6

DataFrame after adding new column 'C':

	A	B	C
0	1	4	7
1	2	5	8
2	3	6	9

Inserting a New Column

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})  
print(df)
```

```
df.insert(1, 'D', [10, 11, 12])
```

```
print("DataFrame after inserting column 'D' at position 1:")  
print(df)
```

OUTPUT

Original DataFrame

	A	B
0	1	4
1	2	5
2	3	6

DataFrame after inserting column 'D' at position 1:

	A	D	B
0	1	10	4
1	2	11	5
2	3	12	6

Replace Contents of Column

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})  
df['A'] = [10, 20, 30]  
  
print("DataFrame after replacing column 'A':")  
print(df)
```

OUTPUT

```
DataFrame after replacing column 'A':  
   A  B  
0  10  4  
1  20  5  
2  30  6
```

Deleting Columns

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]})  
print("Original DataFrame:", df, sep='\n')
```

```
df = df.drop(columns=['A', 'B'])  
print("DataFrame after deleting columns 'A' and 'B':")  
print(df)
```

OUTPUT

Original DataFrame:

	A	B	C
0	1	4	7
1	2	5	8
2	3	6	9

DataFrame after deleting columns 'A' and 'B':

	C
0	7
1	8
2	9

REMOVING ROWS FROM A DATAFRAME

```
df = pd.DataFrame({'A': [1, 2, 3, 4, 5], 'B': [4, 5, 6, 7, 8]})
```

```
# Drop the row with index 3
```

```
result = df.drop(3)
```

```
print("\nAfter dropping the row at index 3:")
```

```
print(result)
```

OUTPUT

Original DataFrame:

	A	B
0	1	4
1	2	5
2	3	6
3	4	7
4	5	8

After dropping the row at index 3:

	A	B
0	1	4
1	2	5
2	3	6
4	5	8

```
df = pd.DataFrame({'A': [1, 2, 3, 4, 5], 'B': [4, 5, 6, 7, 8],  
                  'C': [9, 10, 11, 12, 13]}, index=['r1', 'r2', 'r3', 'r4', 'r5'])
```

Drop the rows by row-labels

```
result = df.drop(['r1', 'r3'])  
print("\nAfter dropping the rows:")  
print(result)
```

OUTPUT

Original DataFrame:

	A	B	C
r1	1	4	9
r2	2	5	10
r3	3	6	11
r4	4	7	12
r5	5	8	13

After dropping the rows:

	A	B	C
r2	2	5	10
r4	4	7	12
r5	5	8	13

```
df = pd.DataFrame({'A': [1, 2, 3, 4, 5], 'B': [4, 5, 6, 7, 8],  
                  'C': [90, 0, 11, 12, 13]}, index=['r1', 'r2', 'r3', 'r4', 'r5'])
```

Dropping rows where column 'C' contains 0

```
result = df[df["C"] != 0]  
print("\nAfter dropping the row where 'C' has 0:")  
print(result)
```

OUTPUT

Original DataFrame:

	A	B	C
r1	1	4	90
r2	2	5	0
r3	3	6	11
r4	4	7	12
r5	5	8	13

After dropping the row where 'C' has 0:

	A	B	C
r1	1	4	90
r3	3	6	11
r4	4	7	12
r5	5	8	13

Filtering out Missing Values

```
data = Series([1, NaN, 2.5, NaN, 6])
```

OUTPUT

```
0    1.0
1    NaN
2    2.5
3    NaN
4    6.0
dtype: float64
```

```
data = Series([1, NaN, 2.5, NaN, 6])
print(data.notnull())
```

```
OUTPUT 0    True
        1   False
        2    True
        3   False
        4    True
dtype: bool
```

Remove NaN values

1. dropna()

```
data = Series([1, NaN, 2.5, NaN, 6])  
c1n_data = data.dropna()
```

OUTPUT

```
0    1.0  
2    2.5  
4    6.0  
dtype: float64
```

2. fillna()

```
filled_data = data.fillna(0)
```

OUTPUT

```
0    1.0  
1    0.0  
2    2.5  
3    0.0  
4    6.0  
dtype: float64
```

Problem Solving

```
data = {
    'Name': ['Amit', 'Ravi', 'Sita', 'John'],
    'Math': [80, np.nan, 75, 90],
    'Sc': [np.nan, 85, 78, 88]
}
df = pd.DataFrame(data)
print(df)
```

OUTPUT

	Name	Math	Sc
0	Amit	80.0	NaN
1	Ravi	NaN	85.0
2	Sita	75.0	78.0
3	John	90.0	88.0

```
print(df.isnull())
```

OUTPUT

	Name	Math	Sc
0	False	False	True
1	False	True	False
2	False	False	False
3	False	False	False

```
print(df.dropna())
```

OUTPUT

	Name	Math	Sc
2	Sita	75.0	78.0
3	John	90.0	88.0

```
df_fill = df.fillna(0)
print(df_fill)
```

OUTPUT

	Name	Math	Sc
0	Amit	80.0	0.0
1	Ravi	0.0	85.0
2	Sita	75.0	78.0
3	John	90.0	88.0

Merging DataFrames

```
pd.merge(df1, df2, on='column_name', how='type')
```

INNER JOIN

```
df1 = pd.DataFrame({  
    'ID': [1, 2, 3],  
    'Name': ['Amit', 'Ravi', 'Sita'] })  
df2 = pd.DataFrame({  
    'ID': [1, 2, 4],  
    'Marks': [80, 85, 90] })
```

```
merg = pd.merge(df1, df2, on='ID')
```

OUTPUT

	ID	Name	Marks
0	1	Amit	80
1	2	Ravi	85

LEFT JOIN

```
pd.merge(df1, df2, on='ID', how='left')
```

OUTPUT

	ID	Name	Marks
0	1	Amit	80.0
1	2	Ravi	85.0
2	3	Sita	NaN

RIGHT JOIN

```
pd.merge(df1, df2, on='ID', how='right')
```

OUTPUT

	ID	Name	Marks
0	1	Amit	80
1	2	Ravi	85
2	4	NaN	90

OUTER JOIN

```
pd.merge(df1, df2, on='ID', how='outer')
```

OUTPUT

	ID	Name	Marks
0	1	Amit	80.0
1	2	Ravi	85.0
2	3	Sita	NaN
3	4	NaN	90.0

Merge DataFrames on Multiple Keys

```
df1 = pd.DataFrame({
    'id': [1, 2, 3, 4, 5],
    'Name': ['Alex', 'Amy', 'Allen', 'Alice', 'Ayoung'],
    'sub_id': ['sub1', 'sub2', 'sub4', 'sub6', 'sub5'] })
df2 = pd.DataFrame({
    'id':[1, 2, 3, 4, 5],
    'Name': ['Billy', 'Brian', 'Bran', 'Bryce', 'Betty'],
    'sub_id':['sub2','sub4','sub3','sub6','sub5'] })

result = df1.merge(df2, on=['id', 'sub_id'])
```

OUTPUT

	id	Name_x	sub_id	Name_y
0	4	Alice	sub6	Bryce
1	5	Ayoung	sub5	Betty

Working with CSV File

A CSV file stores data in the form of **rows and columns**, where:

- Each **row** represents a record
- Each **column** represents a field
- Values are separated by **commas (,)**

Sample CSV File (**emp.csv**)

```
EmpID,Name,Department,Salary,Age
101,Amit,HR,40000,25
102,Ravi,IT,60000,30
103,Sita,Finance,50000,28
104,Neha,IT,65000,32
105,John,HR,45000,26
```

Different Operations on CSV File

Read from CSV File

```
df = pd.read_csv("emp.csv")  
print(df)
```

OUTPUT

	EmpID	Name	Dept	Salary	Age
0	101	Amit	HR	40000	25
1	102	Ravi	IT	60000	30
2	103	Sita	Finance	50000	28
3	104	Neha	IT	65000	32
4	105	John	HR	45000	26

Write to CSV File

```
df.to_csv("file1.csv", index=False)
```

Do NOT save the index column

OUTPUT

```
EmpID,Name,Department,Salary,Age  
101,Amit,HR,40000,25  
102,Ravi,IT,60000,30  
103,Sita,Finance,50000,28  
104,Neha,IT,65000,32  
105,John,HR,45000,26
```


View Last rows

- `df.tail()` → returns last 5 rows (default)
- `df.tail(n)` → returns last n rows

`df.tail(2)`

OUTPUT

	EmpID	Name	Dept	Salary	Age
3	104	Neha	IT	65000	32
4	105	John	HR	45000	26

View First rows

- `df.head()` → returns first 5 rows (default)
- `df.head(n)` → returns first n rows

`df.head(2)`

OUTPUT

	EmpID	Name	Dept	Salary	Age
0	101	Amit	HR	40000	25
1	102	Ravi	IT	60000	30

Selecting a Row

```
df.iloc[0]
```

OUTPUT

EmpID	101
Name	Amit
Dept	HR
Salary	40000
Age	25

Selecting Multiple Columns

```
df[['Name', 'Salary']]
```

View Columns

```
df.columns
```

OUTPUT Index(['EmpID', 'Name', 'Department', 'Salary', 'Age'])

Filtering Data

```
df[df['Salary'] > 50000]
```

OUTPUT

	EmpID	Name	Dept	Salary	Age
1	102	Ravi	IT	60000	30
3	104	Neha	IT	65000	32

Adding New Column

```
df['Bonus'] = df['Salary'] * 0.10
```

OUTPUT

	EmpID	Name	Dept	Salary	Age	Bonus
0	101	Amit	HR	40000	25	4000
1	102	Ravi	IT	60000	30	6000
2	103	Sita	Finance	50000	28	5000
3	104	Neha	IT	65000	32	6500
4	105	John	HR	45000	26	4500

Updating Values

```
df.loc[df['Name']=='Amit', 'Salary']=42000
```

Deleting Columns

```
df.drop('Bonus', axis=1, inplace=True)
```

Deleting Rows

```
df.drop(0) # It removes the row with index 0  
df.drop(0, inplace=True) # It removes the row permanently
```

Sort Data

```
df.sort_values(by='Salary', ascending=False) # Descending
```

Performing Statistical Operations

```
df['Salary'].mean()  
df['Salary'].sum()  
df['Salary'].max()  
df['Salary'].min()
```

Grouping Data

```
df.groupby('Dept')['Salary'].mean()
```

OUTPUT

Dept	
Finance	50000
HR	43500
IT	62500

To give a Statistical Summary of DataFrame

```
df.describe()
```

OUTPUT

	EmpID	Salary	Age
count	5.0	5.0	5.0
mean	103.0	52000.0	28.2
std	1.58	9617.69	2.86
min	101.0	40000.0	25.0
25%	102.0	45000.0	26.0
50%	103.0	50000.0	28.0
75%	104.0	60000.0	30.0
max	105.0	65000.0	32.0

- **std** → Standard deviation (spread of data)
- **25%** → First quartile (25% data below this)
- **50% (median)** → Middle value
- **75%** → Third quartile

Renaming Columns

```
df.rename(columns={'Name': 'Emp_Name'}, inplace=True)
```

Renames column **Name** → **Emp_Name**

Pivot Table

Useful for **grouping** and **aggregating** values

To find average salary per department

```
pd.pivot_table(df, values='Salary', index='Dept', aggfunc='mean')
```

OUTPUT

Dept	
Finance	50000
HR	42500
IT	62500

Join / Merge Operation

Inner Join (*default*)

Used to join two DataFrames based on a common column

df1 =

	EmpID	Name
0	101	Amit
1	102	Ravi
2	103	Sita

df2 =

	EmpID	Salary
0	101	40000
1	102	60000
2	104	70000

```
pd.merge(df1, df2, on='EmpID', how='inner')
```

OUTPUT

	EmpID	Name	Salary
0	101	Amit	40000
1	102	Ravi	60000

Left Join

```
pd.merge(df1, df2, on='EmpID', how='left')
```

OUTPUT

	EmpID	Name	Salary
0	101	Amit	40000
1	102	Ravi	60000
2	103	Sita	NaN

Right Join

```
pd.merge(df1, df2, on='EmpID', how='right')
```

OUTPUT

	EmpID	Name	Salary
0	101	Amit	40000
1	102	Ravi	60000
2	104	NaN	70000

Outer Join

```
pd.merge(df1, df2, on='EmpID', how='outer')
```

OUTPUT

	EmpID	Name	Salary
0	101	Amit	40000
1	102	Ravi	60000
2	103	Sita	NaN
3	104	NaN	70000

Problem Solving

1. Find the highest paid employee

```
high_paid = df[df['Salary'] == df['Salary'].max()]  
print("\nHighest Paid Employee:")  
print(high_paid)
```

2. Find employee with maximum experience

```
max_exp = df[df['Experience'] == df['Experience'].max()]  
print("Employee with Maximum Experience:")  
print(max_exp)
```

3. Count number of employees in each department

```
dept_cnt = df['Dept'].value_counts()  
print("Number of Employees in Each Department:")  
print(dept_cnt)
```

4. Rename columns

```
df.rename(columns={
    'Name': 'Employee_Name',
    'Salary': 'Monthly_Salary'
}, inplace=True)

print(df)
```

5. Delete the 'Experience' column

```
df.drop('Experience', axis=1, inplace=True)
```

Exercises

[Q1 – 16]

Consider following Python dictionary data and Python list labels:

```
data = {  
    'birds': ['Cranes', 'Cranes', 'plovers', 'spoonbills',  
    'spoonbills', 'Cranes', 'plovers', 'Cranes', 'spoonbills',  
    'spoonbills'],  
    'age': [3.5, 4, 1.5, np.nan, 6, 3, 5.5, np.nan, 8, 4],  
    'visits': [2, 4, 3, 4, 3, 4, 2, 2, 3, 2],  
    'priority': ['yes', 'yes', 'no', 'yes', 'no', 'no', 'no', 'yes',  
    'no', 'no']  
}
```

```
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

1. Create a DataFrame birds from this dictionary data which has the index labels.
2. Display a summary of the basic information about birds DataFrame and its data.
3. Print the first 2 rows of the birds Dataframe
4. Print all the rows with only 'birds' and 'age' columns from the Dataframe
5. select [2, 3, 7] rows and in columns ['birds', 'age', 'visits']
6. select the rows where the number of visits is less than 4
7. select the rows with columns ['birds', 'visits'] where the age is missing i.e NaN
8. Select the rows where the birds is a Cranes and the age is less than 4
9. Select the rows the age is between 2 and 4(inclusive)
10. Find the total number of visits of the bird Cranes
11. Calculate the mean age for each different birds in Dataframe.
12. Append a new row 'k' to Dataframe with your choice of values for each column.
Then delete that row to return the original DataFrame.
13. Find the number of each type of birds in Dataframe (Counts)
14. Sort Dataframe (birds) first by the values in the 'age' in descending order, then by the value in the 'visits' column in ascending order.
15. Replace the priority column values with 'yes' should be 1 and 'no' should be 0
16. In the 'birds' column, change the 'Cranes' entries to 'trumpeters'.